

Road Report AI

Team Name: Team 6

Team Members:

Kade O'Bar
Dennis Keithly
Christopher Chaiban
Ben Stidham
Jose Rodas

Supervisor:

Professor Sridhar Alagar

Project Coordinator:

Adarsh Gella

**Erik Jonsson School of Engineering and Computer Science
University of Texas at Dallas**

Date: 5/10/2026

Table of Contents

Executive summary

1. Introduction

2. Project Specification & Management

3. System Architecture & High-Level Design

4. Implementation & AI Orchestration

5. System Performance & Validation

6. Lessons Learned

7. Conclusion & Future Work

• References

• Acknowledgements

Executive summary

Road Report AI is a full-stack road safety platform designed to estimate crash risk for real Texas roads and corridors by combining machine learning, geospatial search, and live contextual data. The system allows a user to search for a road or location, converts that query into coordinates, sends the request to a backend prediction service, and returns a structured safety report with a risk score, tier, supporting breakdown, and route-level context. The project was developed as an AI-orchestrated engineering effort, with the team using an agent-ready specification as the central source of truth for both implementation and coordination.

The final system consists of a React-based frontend and a FastAPI backend connected to a logistic-regression crash risk model. The frontend provides a polished interface for geocoded road search, interactive map visualization, technical methodology, documentation, and backend-connected reporting. The backend handles prediction requests, road suggestions, weather enrichment, scoring logic, and report persistence. Over the course of development, the system evolved from an early prototype into a connected product experience with real backend integration, improved score explainability, weather fallback handling, and a more complete reporting flow.

A major outcome of the project was learning how to balance AI-assisted development with human oversight. The team used structured prompts, technical specifications, and iterative intervention to guide AI-generated implementation while still making key design, architecture, and debugging decisions manually. This workflow accelerated scaffolding, UI iteration, and integration work, but also required careful validation whenever model behavior, frontend-backend contracts, or ambiguous requirements created inconsistencies. As a result, the project demonstrates not only a working technical system, but also a practical example of how AI agents can be orchestrated in a real software engineering process.

Introduction

1.1 Motivation & Problem Statement:

Road safety decisions are often reactive instead of predictive. Emergency responders, transportation planners, and everyday drivers typically have few global resources to estimate road risk. This creates a gap between the large amount of roadway data that exists and the practical ability to use that data to support safer decisions. Road conditions, weather, traffic pressure, and road type all influence crash likelihood, but these factors are difficult to interpret quickly without a structured prediction system.

Road Report AI was motivated by this problem. The goal was to build a system that could transform roadway, environmental, and historical context into a usable crash risk signal for real Texas roads and corridors. Instead of asking users to interpret raw crash records or disconnected map layers, the project aims to provide a simpler output: a road or route search that returns a numerical risk score, tier, and supporting explanation. This makes the data more actionable for users such as first responders prioritizing attention, city planners identifying higher-risk infrastructure, and, in later versions, commuters seeking safer travel awareness.

1.2 Objectives, Scope, and Goals Achieved:

The primary objective of the project was to design and implement a full-stack road safety platform paired with a linear regression model capable of estimating crash risk with the assistance of third party apis for weather and location contexts and historical crash data. Beyond just training a model, the goal was to implement the system in a way that an average commuter could comfortably use it.

The final system includes a React-based frontend, a FastAPI backend, a logistic-regression-based prediction flow supported by geocoded road and location searches and training data generated within the project. Beyond the model additional features such as a methodology page, a documentation page, a connected map experience, and backend support user-submitted roadway issue reports are presented. While more goals were generated during the development process, the core engineering goals were achieved and the system has been deployed.

1.3 Proposed Solution:

The proposed solution was to create a web-based system that combines machine learning, backend API orchestration, and a modern frontend interface to produce crash risk reports for real roads. The system begins with a user entering a road or location query. That query is geocoded into coordinates, which are passed to the backend risk prediction service. The backend then combines model inference with contextual signals such as road class, weather conditions, historical crash patterns, and traffic-oriented heuristics. The result is returned to the

frontend as a structured report containing a crash risk score, severity tier, component breakdown, summary, advice, and related context.

This approach allows the product to function as a cohesive pipeline rather than a collection of disconnected tools. The frontend provides the interaction layer, the backend handles prediction and service orchestration, and the model provides the core scoring intelligence. Supporting systems such as weather enrichment, map rendering, documentation, and methodology presentation make the product more interpretable and usable. In this way, Road Report AI is designed not only to compute a prediction, but to deliver that prediction in a form that supports real decision-making.

1.4 Key Highlights:

The major highlight or aspect of this project is the prediction model that generates risk scores. The model is trained off of TxDOT historic data and combined with Open-Meteo historic weather to accurately provide historical training data. The model is also given real world information on time of day, weather conditions, and user feedback to further improve its predictive ability.

The application designed around this model is also notable as it takes in the outputs from the model and translates them to human readable information. Explanations for the score, breakdowns into how much each part of the score weighted the overall prediction, and a map interface. This map interface takes real time updates and allows for users to spatially organize the riskier roads that exist.

Project Specification & Management

2.1 The Project Specification (The AI's Blueprint + Link):

The project specification for Road Report AI was maintained as an agent-ready Single Source of Truth (SSoT) document that served as the master blueprint for the system throughout development. Rather than functioning only as a traditional requirements document, the SSoT was written to guide both human developers and AI coding agents. It defined the product overview, target users, technology stack, repository structure, API contracts, phase-based milestones, environment variables, non-functional requirements, and engineering constraints. This made it the central reference point for implementation decisions teamwide across the frontend, backend, machine learning pipeline, and supporting documentation.

At a technical level, the specification captured the project as a full-stack road safety platform using a React-based frontend, FastAPI backend, PostgreSQL-compatible persistence, machine learning model integration, weather enrichment, and external mapping/geocoding services. It also documented how the system should evolve over multiple phases, from data preparation and model prototyping to frontend integration, live contextual data, and scalability concerns. Because of this, the SSoT functioned not only as a planning artifact, but also as an operational blueprint for implementation and system validation.

[Road Report AI SSoT](#)

2.2 Maintaining the "Single Source of Truth":

Maintaining the Single Source of Truth (SSoT) was essential to the success of the project because Road Report AI was developed through a combination of human collaboration and AI-assisted implementation. Both AI agents and human contributors depended heavily on explicit, up-to-date instructions. Thus, the SSoT was treated as a living technical contract that needed to remain synchronized with the actual state of the system.

The SSoT was maintained through iterative revision as the project evolved. While two week sections were defined, there were unexpected changes such as the frontend shifted from an earlier frontend that used Vue-based direction to the React-based Sovereign Observer implementation or new api being developed. Thus the specification was updated to reflect those changes. This included revising the product overview, updating the frontend stack, documenting the current repository structure, clarifying implemented and planned API fields, and adjusting milestone descriptions to better match the real state of the application. In practice, the SSoT functioned as both a planning document and a correction mechanism whenever the codebase moved beyond the assumptions of earlier drafts.

Version control practices also supported the maintenance of the SSoT. Changes to the

frontend and backend were tracked in GitHub repositories, while the specification was used to reconcile differences between intended behavior and implemented behavior. Weekly progress reports helped capture what each team member completed, which made it easier to identify where the specification had become outdated and where it needed to be revised. This process was especially important when integrating AI-generated code, since generated output could reflect outdated assumptions if the spec was not kept current.

Overall, maintaining the SSoT required active review rather than passive storage. The document remained valuable because it was continuously updated to reflect real architecture, real endpoints, real frontend behavior, and evolving project priorities. This discipline allowed the project to stay aligned across multiple contributors, changing technologies, and AI-assisted development workflows..

2.3 Key Milestones:

The first major milestone was the transition from project planning into a structured technical foundation. In the early weeks, the team focused on identifying usable TxDOT crash data, researching weather and mapping APIs, outlining the system architecture, and building the first version of the Single Source of Truth (SSoT). This phase established the technical direction of the project and defined future phase goals.

The second milestone was the creation of the first functional frontend and backend scaffolding. During this phase, the team built early UI prototypes, created initial multi-page frontend views, established backend boilerplate in FastAPI, and began shaping the machine learning pipeline and supporting data generation process. This was the point at which the project moved from concept into implementation, with both the user-facing experience and the backend architecture becoming tangible and testable.

A third major milestone was end-to-end integration between the frontend and the backend prediction flow. The team connected geocoded road and location search to the backend risk prediction endpoint, replaced frontend mock data with live API responses, and began rendering real risk scores and related report content in the user interface. This marked the first true MVP stage of the product, where the system could accept user input, perform backend processing, and return meaningful output through the website.

The fourth milestone was the transition to the new React-based Sovereign Observer frontend and the corresponding refinement of the reporting experience. Rather than continuing to iterate on the older Vue implementation, the team established a new primary frontend, connected it to the existing backend flow, added dark mode, improved navigation and browser history behavior, and redesigned the methodology and documentation sections to better reflect the actual system. This phase significantly improved the product's presentation quality while keeping it connected to the real backend.

The final major milestone was backend model and scoring refinement. The team integrated a new logistic regression model into the backend, expanded the backend response schema to include richer scoring information, improved weather handling with caching and fallback logic, and reworked score behavior to reduce clustering and produce more realistic report variation. By the final phase, the project had evolved into a connected full-stack prototype with a functioning frontend, backend prediction pipeline, report generation flow, weather-aware enrichment, and a clearer technical explanation of the methodology behind the scoring system.

2.4 Team Roles & Leader Rotation:

The team used a rotating leadership structure throughout the semester to distribute coordination responsibility and ensure that each member contributed not only technical work, but also project management and communication. Weekly reports show leadership rotating across multiple phases of development, including Dennis Keithly leading during the early planning and research period, Kade O'Bar leading during frontend implementation weeks, Ben Stidham leading during model and integration progress, Christopher Chaiban leading during backend/model transition work, and Jose Rodas leading during the final integration and refinement stage. This rotation helped the team maintain accountability while also allowing different members to guide the project during the phases most aligned with their technical strengths.

Technical responsibilities became more specialized as the project matured. Kade O'Bar focused primarily on frontend design, UI refinement, navigation flow, integration of the new React-based Sovereign Observer interface, and final presentation-facing polish. Ben Stidham contributed heavily to map integration, machine learning experimentation, model development and refinement, and connecting risk outputs to visual map behavior. Jose Rodas took on a major role in full-stack integration by connecting the frontend to the FastAPI backend, expanding API response handling, implementing crash reporting functionality, and improving the system's end-to-end user experience. Dennis Keithly concentrated on data engineering, model training preparation, weather data pipelines, CSV generation and preprocessing, scalability research, and backend scoring improvements. Christopher Chaiban played a central role in maintaining and refining the Single Source of Truth, supporting backend development, coordinating agent-oriented tooling, and helping align deployment and integration changes across the repositories.

Although members had primary focus areas, responsibilities often overlapped when major integration work was needed. Frontend decisions frequently depended on backend contract changes, model work depended on data and weather pipeline improvements, and documentation had to be updated continuously as the system evolved. As a result, the project functioned as a collaborative effort rather than a set of isolated assignments. The rotating leadership model supported this well by allowing different members to coordinate the team during planning, prototyping, integration, and final refinement phases.

System Architecture & High-Level Design

3.1 System Overview:

Road Report AI is a full-stack web platform that estimates roadway crash risk from user-provided locations. The system consists of a React/Vite frontend and a FastAPI backend, with model inference and weather/context enrichment handled server-side. Users search for a road or location, the frontend geocodes that query into latitude/longitude, and the backend returns a structured risk report including a normalized risk score, tier classification, and component-level explainability, and advisory text. At a high level, the architecture includes:

- Frontend (Vercel): User interface for search, map visualization, and safety report presentation
- Backend API (Render): FastAPI service that validates requests, runs risk scoring logic, integrates weather/context data, and returns report payloads
- Data/Model Layer: Logistic regression model artifacts and engineered features used for runtime inference
- Persistence Layer: PostgreSQL-backend storage for user-submitted roadway reports.

High-Level API Specification:

- Base path: /api/v1
- Protocol: HTTPS REST API
- Payload format: JSON
- Validation: Pydantic schemas for request/response contracts
- Error handling: standardized error envelope with code/message/details

Key Endpoint Definitions

- GET /api/v1/health
 - Returns service health and database connectivity status
- GET /api/v1/health/model
 - Returns model readiness and runtime metadata availability
- POST /api/v1/risk/predict
 - Core prediction endpoint. Accepts coordinates and optional context fields, then returns a risk score, total, tier, and component breakdown/summary
- GET /api/v1/risk/model-metrics
 - Returns model metadata
- POST /api/v1/reports
 - Persists user-submitted roadway issue reports

3.2 Architecture Diagrams:

No explicit architectural diagrams have been constructed over the course of this project. Views of data structure can be found inside the [SSoT](#) but otherwise, none exist.

3.3 Key Architectural Decisions and Trade-offs

Unfortunately, the development of the model went on all the way to the end of the project and thus development of the MVP took priority over the ability to scale the model. Despite that, plans to implement multiple instances of the model and create a load balancer to connect all the instances was the plan for model scalability. This would massively inflate running costs of the application, but would allow for much better performance for multiple users. Other choices like caching would help speed up repeated requests, but there are too many roads to prevent massive storage inflation inside the backend.

The other source of speedup in terms of scalability came from the weather api. While not directly implemented inside the current prediction pipeline due to development of the model, scalability was considered. The weather api uses bulk requests and caching. Caching was a valid choice by creating a grid like system that rounded up longitude and latitude coordinates. This created less precise weather coverage but allowed for faster access for repeat requests. Bulk caching was implemented primarily to counteract the issue with free access to Open-Meteo. Without priority access to Open-Meteo via a premium subscription there was no way to speed up access very much. However, bulk requesting allowed for slowdown of requests and thus improved the response rate of the api.

Implementation & AI Orchestration

4.1 Technologies Used:

Frontend Tech Stack:

- React 19
 - Where it fits: Core UI framework
 - Purpose: Builds the single-page landing experience, manages state, and handles user interactions like road searches and report submission
- TypeScript
 - Where it fits: Entire frontend codebase
 - Adds strong typing for API contracts and UI data models, reducing runtime bugs and making backend/frontend integration safer
- Vite
 - Where it fits: Build/dev tooling
 - Purpose: Fast local dev server + optimized production bundling for Vercel deployment
- Tailwind CSS 4
 - Where it fits: Styling layer
 - Purpose: Rapid, consistent UI styling for layouts, spacing typography, component states, and responsive design.
- MapLibreGL
 - Where it fits: Styling layer
 - Purpose: rapid, consistent UI styling for layouts, spacing, typography, component states, and responsive design
- Utility libraries: clsx + tailwind-merge
 - Where they fit: Classname composition helpers
 - Purpose: cleanly build conditional Tailwind class strings without conflicts
- Frontend deployment: Vercel
 - Where it fits: Hosting/runtime for built static SPA
 - Purpose: Serves production frontend and injects environment config used to call backend APIs

Backend Tech Stack:

- Python 3
 - Where it fits: Entire backend runtime and data/model scripts
 - Purpose: Powers API server, model inference service, and preprocessing utilities
- FastAPI
 - Where it fits: HTTP API layer
 - Purpose: Exposes versioned endpoints for health, risk, prediction, weather, and user reports with OpenAPI docs.
- Pydantic v2 + pydantic-settings

- Where it fits: Request/response schemas + environment config
 - Purpose: Validates incoming payloads, serializes consistent responses, and strongly types environment variables
- SQLAlchemy 2.0 (async)
 - Where it fits: ORM/data access
 - Purpose: Manages persistence models, DB sessions, and async transactions
- PostgreSQL
 - Where it fits: Production database
 - Purpose: Stores persistent application data
- PyTorch
 - Where it fits: Machine learning framework
 - Purpose: Runs custom-built machine learning models
- Pandas + Numpy
 - Where they fit: Feature shaping/encoding and numerical processing in inference + CSV/training scripts
 - Purpose: Transform input data into model-compatible tensors and support data-prep workflows
- HTTPX
 - Where it fits: Outbound API calls
 - Purpose: Async HTTP client for weather providers and geocoding-related external calls
- Uvicorn
 - Where it fits: ASGI server
 - Purpose: Runs FastAPI app in production and deployment
- Backend deployment: Render
 - Where it fits: Managed hosting for API service
 - Purpose: Runs backend with env vars, serves public API, and connects to managed Postgres

4.2 AI Build Strategy:

When using cursor for AI-assisted coding, cursor rules was utilized to optimize prompting using aspects from a public github repo (<https://github.com/PatrickJS/awesome-cursorrules>). The .cursorrules is available at the backend github repo. [skill.sh](#) was also utilized for improved agent performance.

4.3 Human-in-the-Loop Interventions

During creation of training data, a lot of different variables in the form of column names and global values were traded between functions and files. While juggling dozens of different data sources the model would either reject certain columns or add in arbitrary columns. Thus, vigilant human oversight and correction would be needed during this process so that it could create expected data in expected locations that could eventually

be used to create training data.

4.4 Codebase:

The project was split into two separate GitHub repositories, one for frontend and one for backend such that deployment would be more efficient and different teams working on different ends of the stack would not accidentally interfere with each other's work. The following READMEs for each end of the full stack application are below.

[Frontend Repo Primary README](#)

[Backend Repo Primary README](#)

In addition to these primary README documents, certain directories have READMEs if their specific technologies or use cases differed or were more specific than the documentation than the individual primary READMEs showed.

For frontend, both the legacy project and the current Sovereign Observer implementation have individual READMEs going over technologies used and how to activate them. The Sovereign Observer implementation is the version of the frontend that is implemented and thus it is mostly overlapping data but is good to have access to. [README](#)

In the backend the csv directory is responsible for creating the training data that is used to run the AI has its own documentation. While not critical for running the code, the pipeline is inside of the backend directory and thus can be referenced. [README](#)

More detailed explanations of the individual functions can be found inside the files through various comments. Additionally, the single source of truth provides good general documentation for what each API does and what technology goes into which part of the project

[Road Report AI SSoT](#)

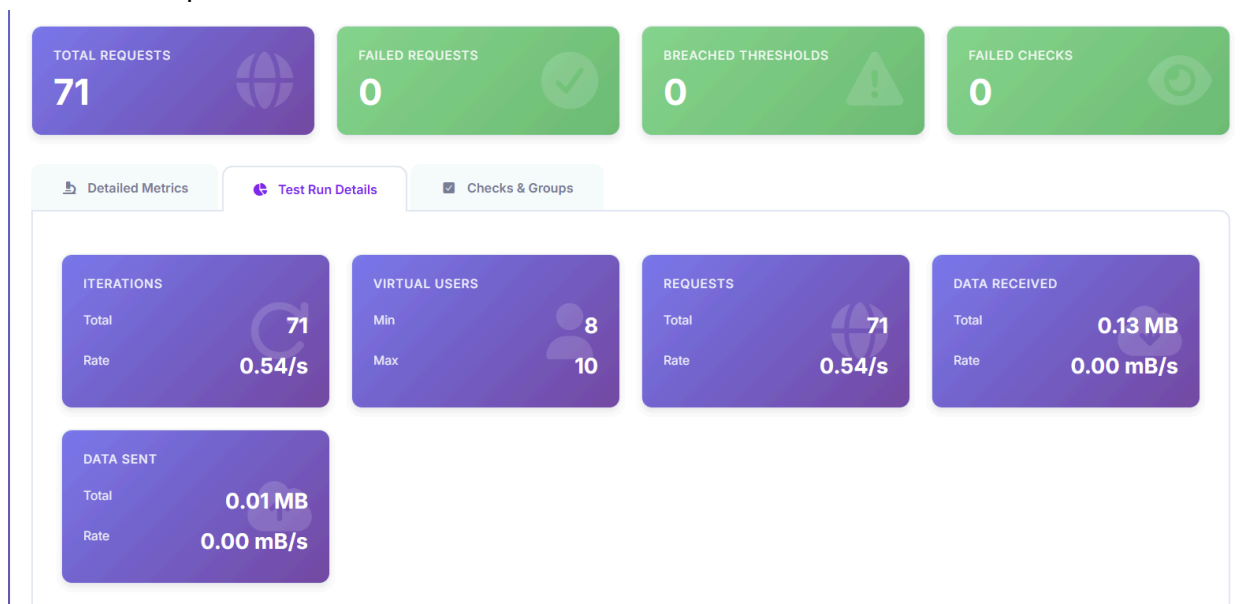
System Performance & Validation

5.1 Testing and Validation:

API endpoints were tested and validated using Swagger UI to ensure that the endpoints worked and produced expected data. This allowed for rapid validation of backend functions. Its main use was checking model output and allowing for different parameters of the request to be changed quickly to view how they affected output.

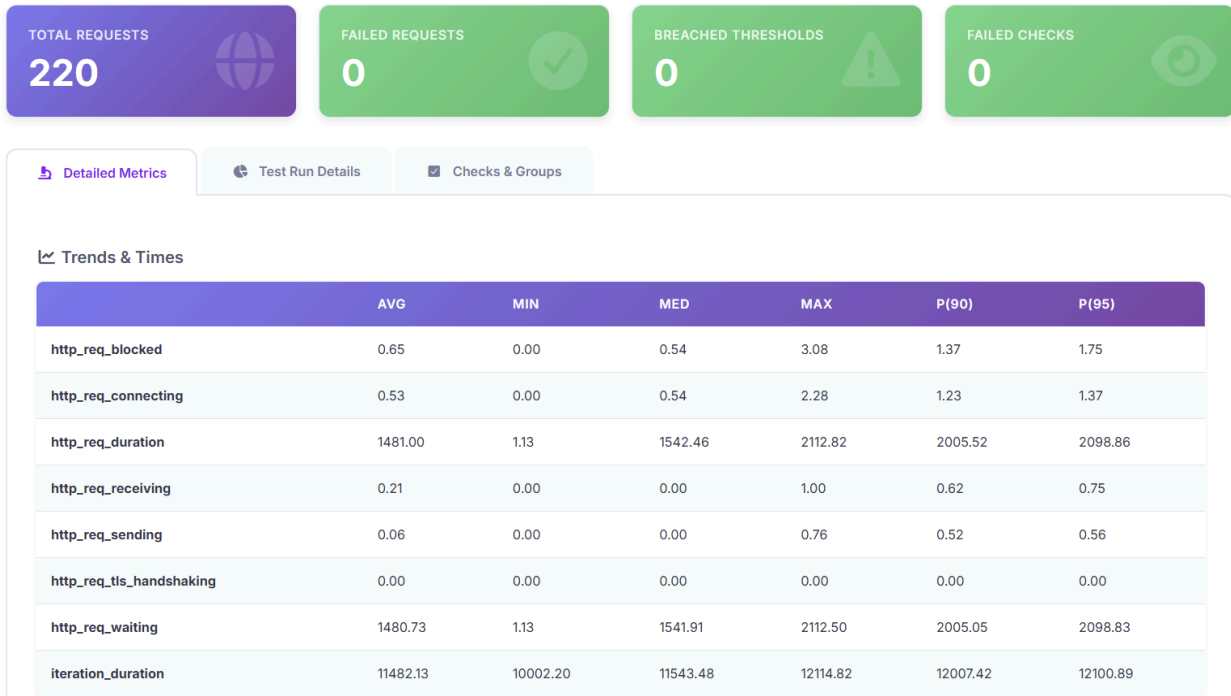
5.2 Technical Performance Metrics:

The scalability of the API and Model was tested using K6 testing. Most scalability was limited due to miscommunication and time constraints. Thus, the standard test was to use 10 virtual users over a period of 2 minutes, with 10 seconds between individual user requests. The overall model performed as such:



This shows high reliability in which no requests were dropped, but one request took around 4 seconds to be responded to. This was primarily due to the fact the caching was impossible given the scope of Texas roads and thus repeat roads would need to be rechecked and only one model was run meaning limited resources existed to speed up requests.

One of the aspects of the model was its ability to fetch weather. While the service to fetch weather is not the weather API which does have some scalable features such as caching and bulk requesting



This shows similar reliability and much higher performance, but not hitting the 240 requests maximum. This performance is likely the limit of how fast a free service can access Open-Meteo data. Hopefully incorporating these strategies into the service would result in a model speedup. 7.2.B has more about future plans for scalability.

5.3 AI Model Performance.

The AI performance, beyond the response time of the AI model, was unable to be measured. Risk scores, the output for the AI model, is a very vague criterion for success. No form of validation was able to be achieved.

Should time have existed to validate the AI, a plan would have been to convert large NOAA files and feed the AI several years worth of training data such that a complete history of patterns could be used. An unknown year or area could then be used to compare results. By having the AI generate results for a known area and then comparing those results to the actual data, general validity could have been checked. An unknown area could also be checked to see if the AI model could get close to accurate results, though this would be harmed by the unknown history of the area.

Both of these were loosely planned around, but creating the data would take several more weeks of work to accomplish.

Lessons Learned

6.1 Insights Gained

This project and the use of AI assistance to primarily generate the code shifted the focus from a specific and high level understanding of the code to requiring a full picture of what a piece of the application needed to run in terms of both data and general flow. While less time was spent working with the code directly, much more time was spent planning each change that needed to be made and understanding every aspect of it before going to the AI to implement the change.

The shift from manual coding to AI assistance also stressed documentation that could be both fed into the AI agent and shared with teammates. As systems evolved, new requirements would be needed. While as teammates we often understood a lot of the surrounding systems, the AI model would not have this context unless asked to find it or provided it. Providing complete documentation of surrounding systems became a priority for all team members so that the agents assisting with code could generate high quality code.

6.2 Lessons from Challenges:

One major challenge encountered during this project was using third party API or information both to gather current day context, display road maps, and provide information for Training Data. These data sources often had different ways of formatting data and sometimes just had similar but different data. One of the challenges was figuring out which source of data had usable data with little need for large amounts of preprocessing. Some of the data was directly observed but unusable given limitations of how much coverage existed while some data was more theoretical but had total coverage.

Another challenge specifically in the realm of AI tools was its difficulty processing and balancing several inputs at the same time. This resulted in the need for much better documentation inside documents like the SSoT and more custom instructions for AI agents. In some cases it required human intervention and the ability for the team to really understand the code and syntax as opposed to just accepting AI output or only having a superficial understanding of what the agent had produced.

6.3 Skills Developed: Assessment of how the team's skills in AI engineering, teamwork, etc.

The specifics regarding using AI in this project can be seen in each member's individual reflection, but the general idea is that AI is a tool that can significantly boost development of products just from an idea. You just have to treat it as such and work alongside it to improve upon its capabilities as well as prevent any

possible mishaps. Regarding teamwork, excluding the weekly meetings with the professor, we had our own weekly meetings to discuss what each member planned to do as well as what they have completed. This helped with improving communication and preventing anyone from being unaware or confused about certain steps with the project. In addition, the rotation of team lead allowed each member to experience leadership and the responsibilities it entails, which allows for growth in any future group projects.

6.4 Individual Reflection:

Kade O'Bar:

Working on this project helped me grow a lot in both frontend development and AI-assisted software engineering. Most of my contribution centered around the frontend, especially improving the design, navigation, report flow, documentation, and overall user experience. Over time, I also had to focus more on making sure the frontend stayed aligned with the backend and the latest model integration, which taught me that a polished interface is only valuable if it is connected to real functionality.

One of the biggest things I learned was that AI tools can speed up development, but they still need a lot of human direction. I found that AI was helpful for iteration and implementation, but not always for making good product decisions or keeping the system consistent. A lot of my work involved reviewing, correcting, and refining generated output so that it actually fit the project. That experience made me more confident in using AI as a tool while still relying on my own judgment to guide the final result.

Overall, this project helped me improve not just technically, but also in how I think about building software as part of a team. I got better at adapting to changing project needs, connecting design decisions to backend functionality, and contributing to something that had to work as a complete system instead of just a standalone interface.

Dennis Keithly

I found that this project has helped me in terms of my ability to reason and work through a problem. Needing to be able to view an entire problem at once and have a general workflow that can be fed into an AI agent so that it can implement the flow has taught me about thinking ahead to a larger degree. Given that my role was primarily data engineering and generation which had several pipelines and moving parts, the ability to think ahead was something critical to me.

I have found that ensuring my thought process was sound, even outside of data generation, has shown me how worthwhile documentation of thought process is on a large scale.

Having documents that I can look back on helped when I needed to shift priorities for a couple of weeks. This allowed me to very quickly pick up where I left off and continue the work that I had dropped off.

One of the main problems I had or lessons that I would take away is that it is always good to be looking farther ahead than the next immediate problem. In data generation I spent weeks working with NOAA weather data and going through very complicated steps to process this data into a format that would be workable. I made many unideal assumptions about the weather and scale of the world on top of this. Eventually I hit a dead end and needed to switch to Open-Meteo for weather which not only removed the unideal assumptions but resulted in a much faster and more efficient weather pipeline for training data. Looking ahead farther and stress testing tools before development is a hard lesson that I have taken from this project.

Christopher Chaiban

This is one of the first projects where the use of AI was pushed for rather than scrutinized, and because of that, I felt that it gave me a better idea of what the landscape of Computer Science/Software Engineering is beginning to lean towards. Rather than blindly using whatever AI tools I find and hoping they produce a somewhat acceptable product, I learned about the various external tools like skills.sh and cursor rules along with other ones that can turn the conversation between myself and the AI into something more professional.

And as such, some challenges would arise due to the new component I was not entirely familiar with. It took some trial and error to get used to the new features and finding all that was available in the first place. This was also my first time dealing with frontend/backend so some extra learning was necessary to gain a grasp on what I was dealing with even if the AI was doing most of the heavy lifting.

Ben Stidham:

I've worked with machine learning models, and in their development before, but not to this extent. Working through the numerous parameters, with what affects the risk score the most, was interesting. Similarly to some of the others, this was also one of the first projects I've done where AI use was heavily encouraged. This shifted how my development was done, mostly just making it more efficient.

Some of my main challenges were bugs with the model, its integration, and the maps view. The maps had enough issues at the end that they had to be cut from the final product, as they failed on deployment but worked locally. Getting the model to predict accurately based on factors like weather and specific roads was an interesting challenge, but required a lot of iteration

Conclusion & Future Work

7.1 Summary of Key Accomplishments

The road report AI most certainly hit a minimum viable product. At this point, the model is working with all of the input it requires, a basic data generation pipeline exists, a basic map view that reacts to AI output exists, and user reporting exists.

The initial goal was just to create the model which has been completed. The model is trained on real data, and has been seen to have variable output depending on the road type and location requested. The model is capable of being given a set of latitude and longitude coordinates and then working out a risk score based off of data such as training data history in regards to city, county, date, and time, as well as real world weather.

The data that the road access is not only easily visualizable through a map for the average user, but the aspects that created the risk score are also visible to the user. Furthermore, while not in the original scope of the project, user reported incidents are also factored into the risk score generated and are reflected across the map.

These satisfy the initial goal which was to provide a basic view of the potential risks associated with driving through specific roads.

7.2 Future Enhancements:

7.2.A Training Data Scope

One of the initial selling points of the project was its ability to be scaled to all of Texas and possibly anywhere at all. While the AI has the ability to take in information from practically anywhere and return a risk score to the user, the data that it has been trained on does not support it. The data generation pipeline would need to be expanded to complete this task.

To expand upon the Texas data, a preprocessing pipeline would need to be created for TxDOT's Crash Report Information System bulk files. These are large directories that are able to report all the crashes within a year. These crashes and their assorted data would need to be mapped out and combined so that the existing weather data pipeline could create usable negative samples.

This would produce usable data, but with the current data generation pipeline expanding the size of the initial data by around 60 times this would result in multiple terabytes of data. Thus, a system would need to be designed to limit the amount of negative samples generated.

The result of this would allow for us to have multiple years of crash data and thus reach some sort of acceptance testing. An untrained year could have training generated for it and we use the Ai to see if it can calculate how many crashes exist in a given year, though a process would need to be designed to see how to translate risk scores with total crashes.

7.2.B Scalability

Scalability, while a topic that came up during the creation process of this application, was not one that was implemented fully. This, along with the data generation pipeline would be one of the first things that should be accomplished if this project were to continue.

While a weather api exists that is scaled as well as it can be without a paid access to Open-Meteo servers, the weather service that the model uses does not yet implement this api. One of the quickest fixes to improve scalability would be to link these two services together that way weather lookup would not slow down the model any more than it already is. It would be ideal to have paid access such that the next part of implementing scalability would be easier.

The most efficient way to scale the model would be to have multiple instances of the model running and have a load balancer direct traffic. Unfortunately, unlike the weather API, there are just too many road instances to reliably cache, though some algorithm could be designed to figure out which roads get queried most often and then refetch those roads periodically and store the resulting info. However the latter would likely require real users in order to ensure that systems are efficiently running.

7.2.C Additional Features

These are much lower priority than scalability and better data pipelines. Should the project continue, these should be done last. More features can always be added to the list, these are suggestions or thoughts the team has had.

For the average user, one who wants a safe route from points a to b, some navigation functionality could be added to the application, or some way of querying multiple paths at once. This is beyond the scope of what the group initially set out to solve though.

More map interactions would also help the average use case. Ideally clicking on a road or making it easier to query the AI from the map would be ideal. This may result in a new map application needing to be created and thus the feasibility of this addition is not guaranteed.

References

<https://skills.sh/>

Acknowledgements

The Road Report AI team would like to thank Doctor Alagar and the TA Adarsh for their assistance and overview of this project. We appreciate their time and the resources they pointed us toward as well as their feedback. This project would not have been able to be done without your assistance.